

PLATFORMĂ CLOUD-NATIVĂ • MICROSOFT AZURE

Reparo marketplace de meseriași.

O soluție SaaS de conectare a clienților cu meseriași independenți, construită integral pe Microsoft Azure.

ECHIPĂ DE PROIECT • 4 MEMBRI

M1 Rosu Ioan

M2 Antoniev Gabriel

M3 Mazilu Cosmin Alexandru

M4 Mardare Andrei Daniel

PROBLEMA REALĂ

O piață de meseriași *fără* infrastructură digitală.

Milioane de contractori independenți operează exclusiv prin telefon, WhatsApp și recomandări orale. Nu există rezervare, confirmare automată, trasabilitate sau CRM propriu. Soluțiile tradiționale eșuează pe cost și pe specializare.

SOLUTII ACTUALE

Telefon. WhatsApp. Confirmări verbale.

- ✗ DESCOPERIRE
Recomandări orale, anunțuri pe stâlpi, grupuri Facebook nemoderate.
- ✗ REZERVARE
Apeluri telefonice, mesaje, confirmări verbale → neînțelegeri și neprezentări.
- ✗ CRM
Contractorul nu știe câte rezervări are; status manual, fără trasabilitate.
- ✗ SOLUȚII CLOSED
OLX / Facebook = panouri de anunțuri; Thumbtack / Angi = comisioane 20–40%, indisponibile RO/EE.
- ✗ SOLUȚII CUSTOM
Servere, licențe SQL, IT dedicat — Investiție upfront nejustificată la început.

REPARO

Marketplace + CRM + AI, livrate ca serviciu.

- + PLATFORMĂ
Multi-tenant pe Microsoft Azure, arhitectură event-driven, servicii managed.
- + INFRASTRUCTURĂ
Zero on-premises: tot rulează ca App Service, Static Web Apps, Functions Consumption.
- + SCALABILITATE
Azure SQL + Functions se scalează automat cu traficul, fără supraprovizionare.
- + COST
Strict Operational Expenditure: vCore-secunde, GB Blob — zero costuri fixe de hardware.
- + VELOCITATE
Echipă de 4, buget de student \$100 credite, MVP funcțional în 5 faze de dezvoltare.

ELEMENTUL DE INOVAȚIE

Inovație · Marketplace + mini-CRM *într-o* singură platformă.

Elementul de inovație al aplicației **Reparo** constă în integrarea, într-o singură platformă, a unui marketplace pentru clienți cu un mini-CRM dedicat contractorilor independenți. Spre deosebire de soluțiile clasice de listare a serviciilor, Reparo nu se limitează la afișarea unor profiluri, ci automatizează fluxul complet dintre client și contractor: autentificare, profil public, filtrare, consultare prin chatbot FAQ, creare booking, notificare asincronă și gestionarea statusului cererilor.

În plus, aplicația integrează un chatbot bazat pe **Gemini API**, care permite clientului să obțină răspunsuri rapide înainte de a face o rezervare. Această funcționalitate reduce numărul de întrebări repetitive adresate contractorului și îmbunătățește experiența de utilizare.

ANALIZA SOLUȚIILOR EXISTENTE

Marketplace public,și CRM privat,și AI — într-un singur produs.

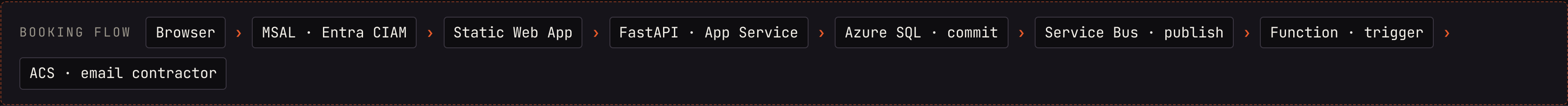
CRITERIU	REPARO	OLX / FB MARKETPLACE	THUMBTACK / ANGI	HUBSPOT / TRELLO / NOTION
Marketplace public	DA • GRILĂ FILTRABILĂ	DA	DA	NU
Booking + status	DA • 4 STĂRI	NU	DA	PARȚIAL
Notificări automate	EVENT-DRIVEN • ACS	NU	DA	NU
Chatbot AI pe profil	GEMINI API	NU	NU	NU
CRM contractor	DA • DASHBOARD	NU	PARȚIAL	DA
Cost intrare	GRATUIT • OPEX	GRATUIT	COMISION 20-40 %	MANUAL SETUP



ARHITECTURA SISTEMULUI

Organizare pe straturi · separare clară a responsabilităților.

STRAT 01 Prezentare	Azure Static Web Apps React 19 + Vite 8 React Router v7 @azure/msal-react 5.3	
STRAT 02 Identitate	Microsoft Entra External ID Microsoft Graph API OIDC Discovery JWT RS256 · JWKS	
STRAT 03 Backend API	Azure App Service FastAPI + Uvicorn SQLModel ORM python-jose · httpx	
STRAT 04 Persistență	Azure SQL Database Azure Blob Storage ODBC Driver 18 user · contractorprofile · booking	
STRAT 05 Async & Email	Azure Service Bus Azure Functions · Consumption Azure Communication Services queue booking-events	
STRAT 06 AI	Gemini API (Google) gemini-2.5-flash model google-generativeai SDK (sau REST · API Key) prompt engineering · system instructions	



DETALII DE IMPLEMENTARE

Stack tehnologic · de ce *aceste* tehnologii.

FRONTEND / BACKEND

React 19 + Vite 8 SPA · BUILD ULTRA-RAPID	Variabilele VITE_ injectează dinamic configurația publică (Client ID, API URL) în bundle-ul JS la build time. Permite aceluiași cod frontend să ruleze flexibil în medii diferite (local vs. producție) fără modificări în sursă.
@azure/msal-react 5.3 AUTH FLOWS	Bibliotecă oficială Microsoft · acquireTokenSilent, redirect flows, sessionStorage — zero cod custom de autentificare.
React Router v7 ROUTING DECLARATIV	Lazy route protection cu ProtectedRoute · navigare client / contractor fără reload de pagină.
Axios 1.15 HTTP CLIENT	Interceptori pentru atașarea automată a token-ului Bearer la fiecare request.
FastAPI · Python BACKEND FRAMEWORK	Async nativ, OpenAPI/Swagger automat, DI idiomatic (Depends()), validare Pydantic — LOC redus față de Django, tipizare strictă.
SQLModel + python-jose ORM + JWT	SQLAlchemy + Pydantic într-un singur model · RS256 cu JWKS pentru validare token Entra · httpx pentru Microsoft Graph API (fetch email real din CIAM) · pymssql pentru Azure SQL.

9 SERVICII CLOUD

Entra External ID CIAM · MANAGED	Zero cod de gestionare parole; 50.000 MAU gratuit/lună · paradigma „nu construi ce poți consuma ca serviciu”.
Azure SQL Database RELAȚIONAL MANAGED	Backup automat, HA 99.99 % SLA, patch-uri automate · Free Offer 100.000 vCore-secunde/lună — viabilitate cost-zero pentru MVP.
Azure Blob Storage PROFILE-PICTURES	Scalare la infinit, CDN-ready, URL permanent per blob — elimină necesitatea unui file server propriu cu RAID/backups manuale.
Azure Service Bus BROKER MESAJE	At-least-once delivery, DLQ opțional, retry automat · decuplează complet FastAPI de funcția de email.
Azure Functions CONSUMPTION PLAN	Serverless pur · 0 cost în repaus · 1M execuții gratuite/lună · scalare intrinsecă la zero.
ACS Email + Gemini API EMAIL · LLM	ACS — domeniu AzComm gratuit, fără reputație SMTP de configurat. Gemini API (Google) — tier gratuit cu modele Flash (Gemini 2.5 Flash), suficient pentru volumul unui MVP; integrare prin SDK Python sau REST direct din backend FastAPI.

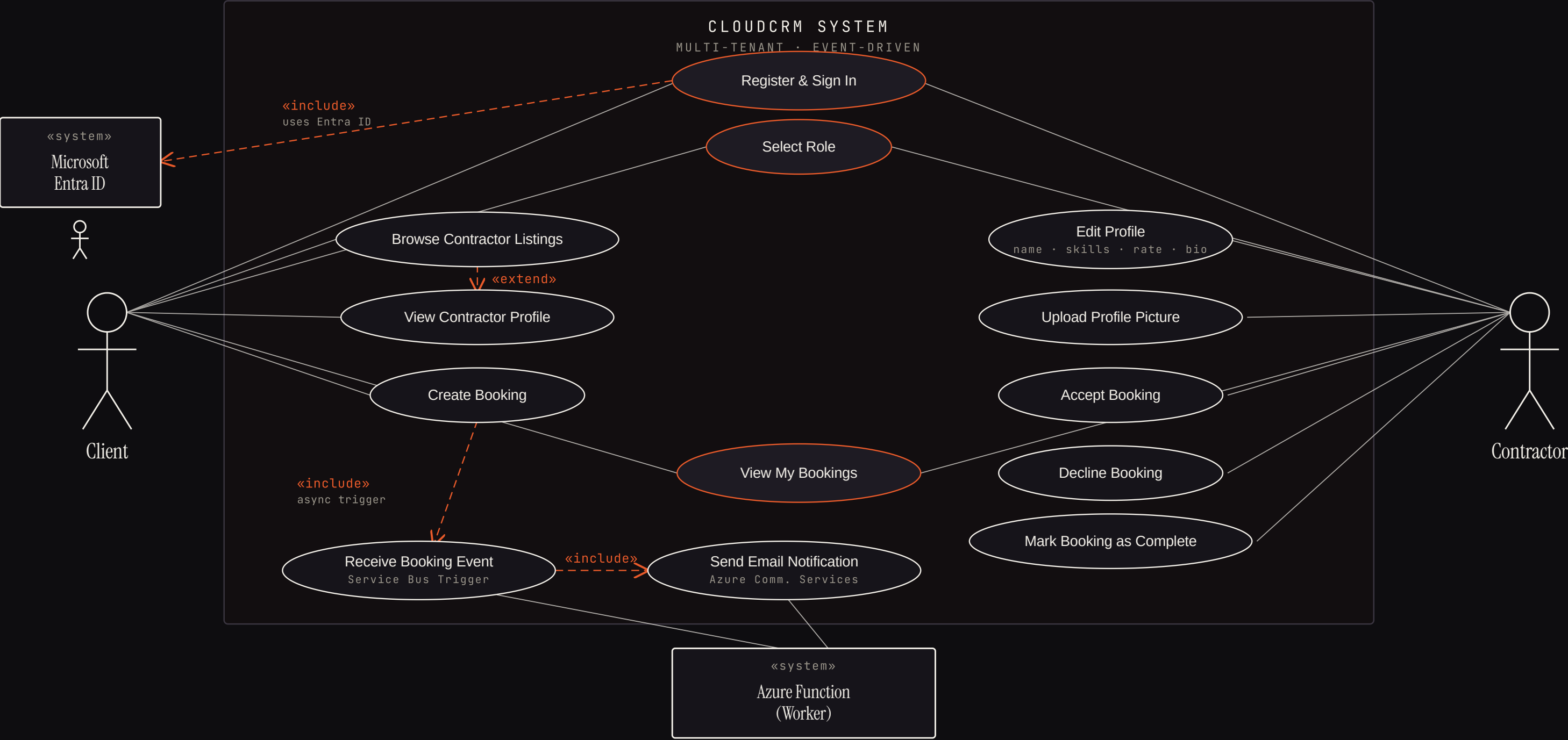
FRAGMENT DE COD • BOOKING EVENT-DRIVEN

Codul care demonstrează *cloud computing* în acțiune.

```
1 @router.post("", response_model=BookingRead)
2 def create_booking(
3     data: BookingCreate,
4     token_payload: dict = Depends(1 verify_token),
5     session: Session = Depends(get_session),
6     publisher: ServiceBusPublisher = Depends(5 get_service_bus_publisher),
7 ):
8     entra_id = 2 token_payload.get("oid") or token_payload.get("sub")
9     client = session.exec(select(User).where(User.entra_id == entra_id)).first()
10    if not client or client.role != UserRole.client:
11        raise HTTPException(status_code=403, detail="Only clients can create bookings")
12
13    contractor = session.get(User, data.contractor_id)
14    if not contractor or contractor.role != UserRole.contractor:
15        raise HTTPException(status_code=404, detail="Contractor not found")
16
17    booking = Booking(
18        client_id=client.id, contractor_id=data.contractor_id,
19        scheduled_at=data.scheduled_at, notes=data.notes,
20    )
21    3 session.add(booking)
22    session.commit() # tranzacție DB confirmată
23    session.refresh(booking)
24
25    event = {
26        "bookingId": booking.id,
27        "contractorEmail": contractor.email,
28        "clientEmail": client.email,
29        "date": booking.scheduled_at.isoformat(),
30        "notes": booking.notes,
31    }
32    4 try:
33        publisher.publish(event, subject="booking.created")
34    except Exception:
35        logger.exception("Failed to publish ...", booking.id)
36
37    return booking
```

- 1 JWT validat ÎNAINTE de business
verify_token descarcă JWKS de la Entra CIAM, găsește cheia RSA după kid și validează semnătura RS256. Token invalid → 401, logica nu se atinge.
- 2 oid = liantul cloud ↔ DB
Identitatea utilizatorului vine din token-ul Entra (Object ID persistent), nu din cookie sau sesiune. Persistă peste re-autentificări.
- 3 Commit DB înainte de publish
Booking-ul e mai întâi salvat tranzacțional în Azure SQL, abia apoi se încearcă publish pe Service Bus. Dacă bus-ul cade, booking-ul există. Ordine inversă = inconsistență de date.
- 4 Fire-and-forget conștient
Erorile de publish sunt logged, nu propagate. API-ul returnează booking-ul indiferent de starea bus-ului. Disponibilitatea notificărilor ≠ disponibilitatea API-ului.
- 5 Lazy singleton via DI
Instanța ServiceBusPublisher e creată o singură dată la primul request, niciodată la import time. Lipsa unei connection string nu crash-uește aplicația la pornire.

Actori, roluri și fluxuri



— ASSOCIATION -.-.- <<INCLUDE>> / <<EXTEND>>

Două roluri • două flow-uri un *singur* SPA.

ROL • CLIENT

Caută. Întreabă. Rezervă.

A	Autentificare via Entra CIAM Sign-up / sign-in redirect flow MSAL • pagina <code>SelectRole</code> la prima accesare.
M	Marketplace cu filtrare live <code>/client/home</code> • grilă contractori • filtre Electrical • Plumbing • Carpentry • HVAC • Painting • General.
P	Profil detaliat contractor <code>display_name</code> • <code>skills</code> • <code>hourly_rate</code> • <code>bio</code> • poza de profil (URL Blob Storage).
?	Gemini înainte de booking <code>FaqChatbot</code> apelează Google Gemini API.
B	Rezervare cu dată, oră, notițe POST <code>/api/bookings</code> declanșează automat notificarea contractorului.
L	Vizualizare rezervări proprii GET <code>/api/bookings/mine</code> • statusuri: pending • confirmed • completed • cancelled.

ROL • CONTRACTOR

Gestionează. Acceptă. Livrează.

D	Dashboard personalizat <code>/contractor/dashboard</code> • tabel cu toate rezervările • email client • dată • notițe • status curent.
±	Acțiuni pe rezervări Accept • Decline • Complete → PATCH <code>/api/bookings/{id}/status</code> • autorizare per contractor.
E	Editor profil integrat PUT <code>/api/contractors/me</code> • update <code>display_name</code> • <code>skills</code> • <code>hourly_rate</code> • <code>bio</code> • <code>profile_image_url</code> .
@	Notificare email automată La fiecare booking nou → email HTML via ACS cu detalii complete • fără nicio acțiune manuală.
S	Stare lucrări în timp real Câmpul <code>status</code> + <code>created_at</code> formează audit log implicit pentru fiecare rezervare.

De la *investiție* fixă la *consum* elastic.

CAPEX ELIMINAT • TRADIȚIONAL

Server fizic • licențe • echipă IT

Server / VM dedicat	3.000–8.000 €
Licențe SQL Server	2.000–5.000 €/an
IT dedicat • backup • patch-uri	fix/lună
Email server propriu / SendGrid	fix
Licențe auth (LDAP, Okta etc.)	fix

>

OPEX ACTUAL REPARO

Plătești doar ce consumi.

Static Web Apps	Free tier
App Service F1	Free • 60 CPU-min/zi
Entra External ID	50.000 MAU/lună
Azure SQL Free Offer	100.000 vCore-s/lună
Blob Storage	~\$0.02/GB/lună
Service Bus Basic	cenți / 10.000 ops
Functions Consumption	1M exec/lună incluse
ACS Email	Free tier suficient

Total MVP în producție ușoară ≈ < \$5/lună

- PAIN RELIEVERS • CENTRIC VPC

— Elimină coordonarea manuală prin WhatsApp/telefon pentru rezervări.

— Elimină absența confirmărilor — ACS notifică contractorul instant.

— Elimină bariera digitală pentru meseriași — profil în 5 minute, fără cunoștințe tehnice.

— Elimină nepotrivirile de așteptări — chatbot FAQ calificator reduce booking-urile nepotrivite.

+ CUSTOMER GAINS • CENTRIC VPC

— Clienți: meseriaș verificat, interogare AI înainte de contact, rezervare în 3 click-uri.

— Contractorii: CRM lightweight în browser, notificări automate, status în timp real.

— Profil digital cu fotografie + bio + tarif afișat — prezență fără efort de marketing.

— Trasabilitate completă: audit log implicit prin câmpurile status + created_at.

BUSINESS MODEL CANVAS



PROCESUL DE DEZVOLTARE

5 probleme concrete.

PROBLEMĂ • 01 Token JWT din CIAM fără claim de email Entra External ID (CIAM) nu include întotdeauna adresa de email în token-ul JWT returnat după autentificare — o restricție de privacy specifică CIAM, diferită de Azure AD clasic. Aplicația nu putea ști cui să trimită notificări. SOLUȚIE ADOPTATĂ Backend-ul obține un token propriu prin Client Credentials și interoghează Microsoft Graph API (GET /users/{oid}) pentru a extrage email-ul real al utilizatorului la prima logare. Email-ul este salvat în baza de date. ALTERNATIVĂ RESPINSĂ Custom claim direct în user flow-ul din Entra — mai simplu de implementat, dar inflexibil și dependent de configurația specifică a tenant-ului.	PROBLEMĂ • 02 Driver SQL incompatibil cu mediul Azure Driverul standard Python pentru SQL Server (pyodbc) nu este un driver pur Python — el depinde de un program extern instalat pe sistemul de operare (Microsoft ODBC Driver 18). Pe serverele Azure, noi controlăm aplicația, nu OS-ul, deci apt-get install nu este o opțiune. Aplicația crăpa la pornire cu erori de driver lipsă. SOLUȚIE ADOPTATĂ Am înlocuit pyodbc cu pymssql — un driver 100% Python care implementează protocolul de comunicare cu SQL Server direct, fără nicio dependență de sistem. Se instalează cu pip ca orice alt pachet Python și funcționează instant pe orice mediu.	PROBLEMĂ • 03 Latență la validarea fiecărui token JWT La fiecare request autentificat, middleware-ul de securitate descarcă cheile publice criptografice de la Microsoft (JWKS) pentru a verifica semnătura token-ului — un apel HTTP extern de ~200 ms care se adăuga la fiecare operație din aplicație. SOLUȚIE PROVIZORIE Acceptat ca limitare pentru MVP — garantează că cheile sunt mereu la zi. SOLUȚIE PRODUCȚIE Cache în memorie cu TTL de 24 h și refresh automat la invalidarea semnăturii — pattern deja implementat în graph.py pentru token-urile Graph API.	PROBLEMĂ • 04 Retry loop infinit la erori de throttling email Când serviciul de email (ACS) returna eroarea 429 (prea multe request-uri), funcția Azure eșua. Service Bus interpreta eșecul drept „mesaj neprocesat” și re-livra instant același mesaj — declanșând din nou funcția, care eșua din nou. Bucla consuma cotele Azure exponențial. SOLUȚIE ADOPTATĂ Am implementat fault-tolerant exception handling în worker: erorile de tip 429 sunt prinse explicit, mesajul este marcat ca procesat (nu re-livrat), și eroarea este logată. Mesajele care eșuează repetat din alte motive ajung controlat în Dead Letter Queue (DLQ).	PROBLEMĂ • 05 Server web implicit Azure incompatibil cu FastAPI După deployment, toate apelurile API returnau 405 Method Not Allowed deși codul era corect. Azure, nefiind instruit explicit cum să pornească aplicația, lansa automat Gunicorn cu un profil WSGI (sincron). FastAPI este un framework ASGI (asincron) — cele două protocoale sunt incompatibile, iar rutele nu erau recunoscute. SOLUȚIE ADOPTATĂ Am configurat explicit comanda de pornire a serverului: python -m uvicorn app.main:app --host 0.0.0.0 --port 8000. Uvicorn este serverul ASGI nativ pentru FastAPI. Imediat după setare, toate rutele au funcționat corect.
--	---	--	--	---

TRANSPARENȚĂ TEHNICĂ

Ce *nu* face încă · și ce a făcut AI în dezvoltare.

LIMITARE · 01

Deploy exclusiv manual · lipsă CI/CD

Codul este împins în Azure manual via CLI (az webapp deploy). Nu am investit timp într-un pipeline GitHub Actions pentru automatizare completă.

LIMITARE · 02

JWKS fetch non-cached

verify_token descarcă JWKS la fiecare request HTTP — latență adițională constantă. TTL-cache 24h ar elimina-o.

LIMITARE · 03

Single region deployment

Toate serviciile trăiesc într-un singur datacenter (PoLand Central). Lipsa de geo-redundancy o face vulnerabilă la picări regionale.

LIMITARE · 04

Rol setat o singură dată

La sign-up, SelectRole.jsx apelează POST /api/users/me. Nu există mecanism de schimbare ulterioară — simplificare deliberată MVP.

LIMITARE · 05

Listare fără paginare

GET /api/contractors returnează toți contractorii fără LIMIT/cursor. Acceptabil la demo · degradează la >1000 contractori.

LIMITARE · 06

Lipsă full-text search

Căutarea contractorilor se face prin queries SQL simple. Lipsa unui motor de căutare avansat (ex. Azure AI Search) limitează flexibilitatea căutărilor pe baza bio-ului.

AI ÎN PROIECT

Arhitectura a fost gândită de echipă · AI a fost o unealtă de asistență.

△ CE A FĂCUT AI

- △ Research și validare rapidă privind capabilitățile SDK-urilor Azure.
- △ Asistență la debugging și parsarea erorilor obscure de infrastructură (CORS, incompatibilități de drivere OS).
- △ Pair-programming pentru deblocarea problemelor specifice (ex. parsing criptografic JWKS).
- △ Generare de snippet-uri pentru integrarea API-urilor externe.

○ CE A RĂMAS ECHIPEI

- Proiectarea arhitecturii Cloud și validarea fluxurilor de date între cele 9 servicii.
- Structurarea modulară a codului (designul arhitecturii backend pe entități, nu ca monolit).
- Implementarea logicii de business, a securității și deciziile de mitigare a riscurilor (ex. trecerea de la pyodbc la pymssql).
- Planificarea etapelor de dezvoltare și adaptarea cerințelor tehnice la bugetul MVP-ului.

users			^
GET	/api/users/me	Get Me	🔒 ▼
POST	/api/users/me	Upsert Me	🔒 ▼
contractors			^
GET	/api/contractors	List Contractors	🔒 ▼
GET	/api/contractors/{contractor_id}	Get Contractor	🔒 ▼
PUT	/api/contractors/me	Update My Profile	🔒 ▼
bookings			^
POST	/api/bookings	Create Booking	🔒 ▼
GET	/api/bookings/mine	Get My Bookings	🔒 ▼
PATCH	/api/bookings/{booking_id}/status	Update Booking Status	🔒 ▼
uploads			^
POST	/api/upload-profile-picture	Upload Profile Picture	🔒 ▼
chat			^
POST	/api/chat/ask	Ask	🔒 ▼
default			^
GET	/	Read Root	▼